SAD

Analysis and Design of a System, documenting and evaluating the system, Data Modelling, Development of Information Management System, Implementation, Testing and Security Aspects.

Chapter - 1

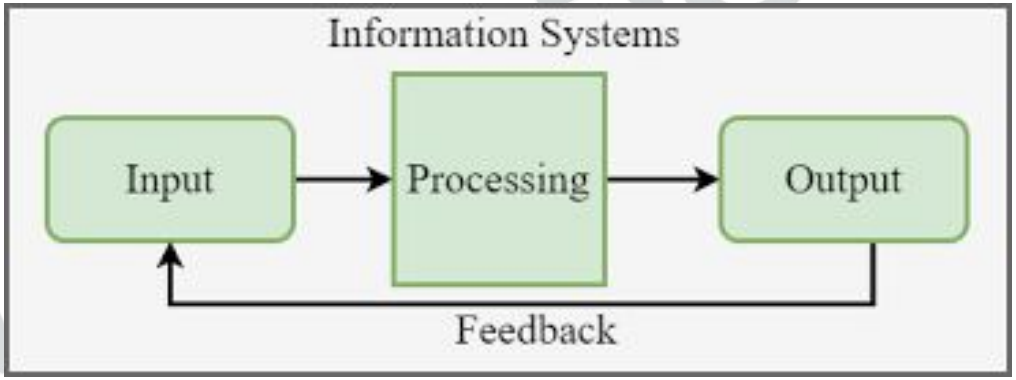
Analysis and Design of a System

FUNDAMENTALS OF SYSTEMS

System is a word derived from the Greek word **‘Systema’** which means an organized relationship among components.

A system may be defined as an orderly grouping of interdependent components linked together according to a plan to achieve a specific goal. Each component is a part of the total system and it has to do its own share of work for the system to achieve the desired goal.

An information system is an arrangement of people, data, processes, information presentation and information technology that interacts to support and improve day-to-day operations in a business as well as support the problem-solving and decision-making needs of management and users.



The characteristics of a system are as follows:

- **Organization** implies structure and order. It is an arrangement of components that helps to achieve objectives.
- **Interaction** refers to the procedure in which each component functions with other components of the system.
- **Interdependence** means that one component of the system depends on another component.

The characteristics of a system are as follows:

• **Integration** is concerned with how a system is tied together. It is more than sharing a physical part. It means that parts of the system work together within the system even though each part performs a unique function.

• **Central Objective** is quite common that an organization may set one objective and operate to achieve another. The important point is that the users must be aware about the central objective well in advance.

Important Terms Related to Systems

Purpose, Boundary, Environment, Inputs, and Outputs are some important terms related to systems.

- A system's **purpose** is the reason for its existence and the reference point for measuring its success.
- A system's **boundary** defines what is inside the system and what is outside.
- A system **environment** is everything pertinent to the system that is outside of its boundaries.
- A system's **inputs** are the physical objects and information that cross the boundary to enter it from its environment.
- A system's **outputs** are the physical objects and information that go from the system into its environment.

Classification of Systems

Systems may be classified as follows:

a) Formal or Informal

b) Physical or Abstract

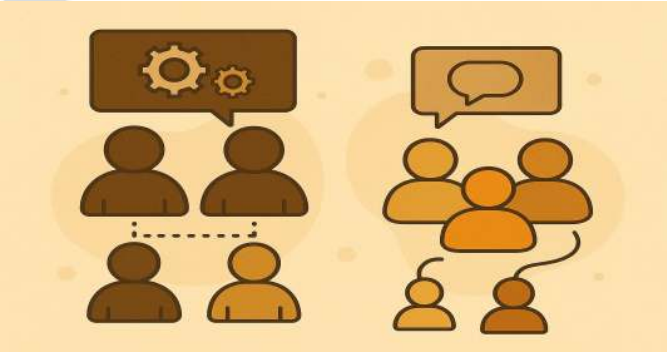
c) Open or Closed

d) Manual or Automated

a) Formal or Informal Systems

A **formal system** is one that is planned in advance and is used according to schedule. In this system, policies and procedures are documented well in advance. A real-life example is to conduct a scheduled meeting at the end of every month in which the agenda of the meeting has already been defined well in advance.

An **informal system** is the system that is not described by procedures. It is not used according to a schedule. It works on a need basis. For example, sales order processing system through telephone calls.



b) Physical or Abstract Systems

Physical systems are tangible entities that may be static or dynamic. Computer systems, vehicles, buildings, etc., are examples of physical systems.

Abstract systems are conceptual entities.
Example: Company

c) Open or Closed Systems

An **open system** is a system within its environment. It receives input from the environment and provides output to the environment.
Example: Any real-life system, information system, organization, etc.

A closed System is isolated from environmental influences. It operates on factors within the system itself. It is also defined as a system that includes a feedback loop, a control element, and a feedback performance standard.

Performance standard is defined as an objective that the system has to meet.

A **feedback loop** is defined as a portion of the system that enables the system to regulate itself. Signals are obtained from the system describing the system status and are transmitted to the control mechanism.

A **control element** compares the output with the performance standard and adjusts the system input accordingly.

d) Manual and Automated Systems

The system which does not require human intervention is called an **automated system**. In this system, the whole process is automatic.
Example: Traffic control system for metropolitan cities.

The system which requires human intervention is called a **manual system**.
Example: Face-to-face information centres at places like railway stations, etc.

REAL TIME SYSTEMS

A real time system describes an interactive processing system with severe time limitations. A real time system is used when there are rigid time requirements on the flow of data. A real time system is considered to function correctly only if it returns the correct result within imposed time constraints. There are two types of real time systems. They are:

- **Hard Real Time Systems** which guarantee that critical tasks are completed on time.
- **Soft Real Time Systems** which are less restrictive type of real time systems where a critical real time task gets priority over other tasks, and retains the priority until it completes them.

Systems that control scientific experiments, medical imaging systems, industrial control systems and some display systems are real time systems.

Select the appropriate choice given under each question.

1. _____ are comprehensive, multiple-step approaches to system development that will guide your work and influence the quality of your final product.

a) Techniques

b) Tools

c) Methodologies

d) Data flows

2. The person in an organization who has the primary responsibility for systems analysis and design is _____.

a) The end user

b) The systems analyst

c) The internal auditor

d) Business manager

3. An overall strategy to information systems development that focuses on the ideal organization data, rather than where and how data are used, best defines the _____.

a) Process-oriented approach

b) Data-organization approach

c) Data-oriented approach

d) Information-oriented approach

4. Which of the following is not a true statement concerning the differences between the process-oriented and data-oriented approaches to systems development?

a) The process-oriented approach has limited design stability

b) Much uncontrolled data duplication exists with the data-oriented approach

c) The data-oriented approach designs data files for the enterprise

d) None of the above

5. Non-information system professionals in an organization who specify the business requirements and who use software applications are called:

a) Programmers

b) Network managers

c) Code designers

d) End users

6. Which of the following is not one of the four classes of information systems?

a) Transaction processing systems

b) Decision support systems

c) Expert systems

d) Production systems

DISTRIBUTED SYSTEMS

A **distributed system** is one in which the **data, process, and interface components** of an information system are distributed to multiple locations in a computer network. Accordingly, the processing workload required to support these components is also distributed across multiple computers on the network.

In this system, each processor has its own local memory. The processors communicate with one another through various communication lines, such as high-speed telephone lines. The processors in a distributed system may vary in size and function.

They may include small microprocessors, workstations, minicomputers, and large general-purpose computer systems.

The implementation of a distributed system is complicated and difficult, but still in demand. Some of the reasons are that modern businesses are already distributed, so they need distributed solutions. In general, solutions developed using a distributed systems paradigm are user-friendly. They have the following advantages:

- Resource sharing
- Computation speedup
- Reliability
- Communication

The Five Layers of Distributed System Architecture Are:

- **Presentation Layer** is the actual user interface. The inputs are received by this layer and the outputs are presented by this layer.

- **Presentation Logic Layer** includes processing required to establish the user interface.
Example: Editing input data, formatting output data.

- **Application Logic Layer** includes all the logic and processing required to support actual business application and rules.
Example: Calculations.

- **Data Manipulation Layer** includes all the command and logic required to store and retrieve data to and from the database.

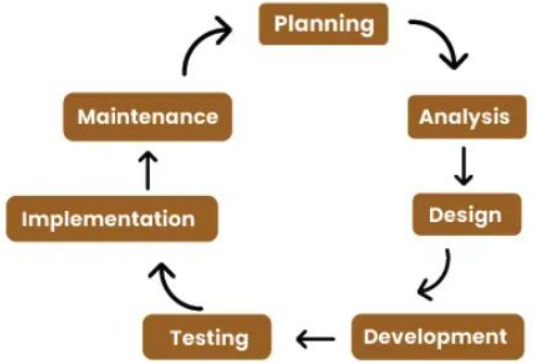
- **Data Layer** is the actual stored data in the database.

DEVELOPMENT OF A SUCCESSFUL SYSTEM

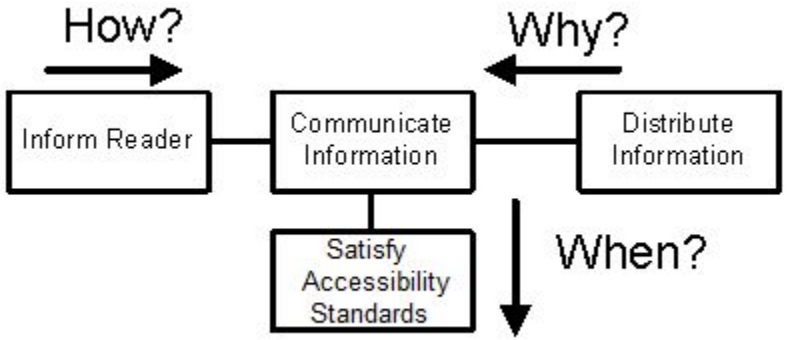
The success of any system depends on the approach of building it. If the development approach is right, the system will work successfully.

System Development Life Cycle.

System development life cycle (SDLC) is a standard methodology for the development of information systems. It mainly consists of four phases: **System Analysis, System Design, System Construction & Implementation, and System Support**. Every phase consists of inputs, tasks, and outputs.



Traditional SDLC was strictly sequential. The developers first complete the previous phase and then start the next phase. But now the concept of **Repository** is introduced in SDLC and it is known as **FAST methodology**, where work is done across a shared repository. It means that all the inputs and outputs of phases must be stored in the repository. At any time, developers can backtrack to the previous phase and they can also work on two phases simultaneously.



For making a successful system, the following principles should be followed:

(1) Both customers and developers should be involved for accuracy in the information.

(2) A problem-solving approach should be adopted. The classic problem-solving approach is as follows:

a) Study, understand the problem and its context

b) Define the requirements of a solution

c) Identify candidate solutions and select the best solution

d) Design and implement the solution

e) Observe and evaluate the solution's impact and refine the solution accordingly

(3) Phases and activities should be established.

(4) For consistent development of a system, some standards should be established.

These standards are:

- **Documentation standards:** It should be an ongoing activity during the system development life cycle.

- **Quality standards:** Checks should be established at every phase for ensuring that the output of every phase meets the business and technology expectations.

- **Automated tool standards:** Hardware and software platforms should be finalized for the development of the information system. Automated tool standards prescribe technology that will be used to develop and maintain information systems and to ensure consistency, completeness, and quality.

(5) Development of an information system should be considered as a capital investment. The developer of an information system should think about several solutions of a particular problem and every solution should be evaluated for cost-effectiveness and risk management. Cost-effectiveness is defined as the result obtained by striking a balance between the cost of developing and operating an information system.Information system and the benefits derived from that system.

Risk management is defined as the process of identifying, evaluating and controlling what might go wrong in a project before it becomes a threat to the successful completion of the project or implementation of the information system.

Multiple feasibility checkpoints should be built into system development methodology. At each feasibility checkpoint, all costs are considered sunk (i.e. not recoverable). Thus, the project should be re-evaluated at each checkpoint to determine if it remains feasible to continue investing time, effort, and resources. At each checkpoint, the developers should consider the following options:

- Cancel the project if it is no longer feasible.
- Re-evaluate and adjust the cost and schedule if project scope is to be increased.
- Reduce the scope if the project budget and schedule are frozen and not sufficient to cover all the project objectives.

(6) Divide and Conquer approach is the way of making a complex problem easier. In this approach, the larger problem (System) is divided into smaller problems (Subsystem).

(7) For development of a successful system, the system should be designed for growth and change. When the system is implemented, it enters the operations and support stage of Life Cycle.

During this stage, the developers encounter the need for changes that range from correcting simple mistakes to redesigning the system to accommodate changing technology to making modifications to support changing user requirements. These changes direct the developers to rework formerly completed phases of the life cycle.

VARIOUS APPROACHES FOR DEVELOPMENT OF INFORMATION SYSTEMS

Various approaches are available for development of Information Systems. They are:

- **Model Driven:** It emphasizes the drawing of pictorial system models to document and validate both existing and/or proposed systems. Ultimately, the system model becomes the blueprint for designing and constructing an improved system.

- **Accelerated approach:** A prototyping approach emphasizes the construction of model of a system. Designing and building a scaled-down but functional version of the desired system is known as **Prototyping**. A prototype is a working system that is developed to test ideas and assumptions about the new system. It consists of

of working software that accepts input, perform calculations, produces printed or display information or perform other meaningful activities.

• **Joint Application Development:** It is defined as a structured approach in which users, managers, and analysts work together for several days in a series of intensive meetings to specify or review system requirements. In this approach, requirements are identified and design details are finalized.

Structured Analysis and Design Approach

The goal of structured system analysis and design is to reduce maintenance time and effort. Modeling is the act of drawing one or more graphical representations of a system. Model-driven development techniques emphasize the drawing of models to help visualize and analyze problems, define business requirements and design information systems. The first model-driven approach is **Structured Analysis and Design approach**.

Structured Analysis is a development method for the analysis of existing manual systems or automated systems, leading to development of specifications (expected functionality or behaviour) for proposed system. The objective of structured analysis approach is to organize the tasks associated with requirement determination to provide an accurate and complete understanding of a current situation. The major tasks of structured system analysis approach are:

- Preliminary Investigation
  - Problem Analysis
  - Requirement Analysis
  - Decision Analysis

It is a process-centered technique that is used to model business requirements for a system.

Structured analysis introduced a process-modeling tool called the **Data Flow Diagram (DFD)**, used to illustrate business process requirements. With the help of DFD, the systems analyst can show the system overview.

Data modeling tools such as **Entity Relationship Diagrams (ERD)** are used to illustrate business data requirements. With the help of ERD, the analyst can show database overview.

Structured Design utilizes graphic description (output of system analysis) and focuses on development of software specifications. The goal of structured design is to lead to development of programs consisting of relatively independent or cohesive modules that perform relatively independent functions. It is a specific program design technique, not a comprehensive design method. Thus, it does not specify file or database design, input or output layout or the hardware on which the application will run. It provides specification of program modules that are functionally independent.

Structure Charts

It is a process-centered technique that transforms the structured analysis models into software design models. Structured design introduces a modeling tool called **Structure Charts**. They are used to illustrate software programs structure to fulfill business requirements. Structure charts describe the interaction between independent modules and the data passed between the modules. These module specifications can be passed to programmers prior to the writing of program code. In structure chart, the whole application is divided into modules (set of program instructions) and modules are designed according to some principles of design. These principles are:

Modularity and partitioning: Each system should consist of a hierarchy of modules. Lower level modules are generally smaller in scope and size compared to higher level modules. They serve to partition processes into separate functions.

Coupling: Modules should be loosely coupled. It means that modules should have little dependence on other modules in a system.

Cohesion: Modules should be highly cohesive. It means that modules should carry out a single processing function.

Span of control: Modules should interact with and manage the functions of a limited number of lower level modules. It means that the number of called modules should be limited (in a calling module).

Size of Module: The number of instructions contained in a module should be limited so that module size is generally small.

Shared use of Functions: Functions should not be duplicated in separate modules; they may be shared. It means that functions can be written in a single module and it can be invoked by any other module when needed.

Prototype

A prototyping approach emphasizes the construction of model of a system. Designing and building a scaled-down but functional version of a desired system is the process known as **Prototyping**. A prototype is a working system that is developed to test ideas and assumptions about the new system. It consists of working software that accepts input, performs calculations, produces printed or displayed information, or performs other meaningful activities. It is the first version or iteration of an information system, i.e., an original model. Customer evaluates this model. This can be effectively done only if the data are real and the situations are live. Changes are expected as the system is used. This approach is useful when the requirements are not well defined. A prototype is usually a test model. It is an interactive process. It may begin with only a few new functions and expanded to include others that are identified later. The steps of the prototyping process are depicted in They are:

- Identify the user's known information requirements and features needed in the system.
- Develop a working prototype.
- Revise the prototype based on feedback received from the customer.
- Repeat these steps as needed to achieve a satisfactory system.

Actual development of a working prototype is the responsibility of a systems analyst. The difference between a prototype model and an actual information system is that a prototype will not include error checking, input validation, security, and processing completeness of a finished application. It will not offer user help as in the final system.

But, sometimes, the prototype can evolve into the product to be built. The prototype can be easily developed with tools of fourth generation languages (4GL) and with the help of Computer Aided Software Engineering (CASE) tools. Prototyping approach is a form of **Rapid Application Development (RAD)**.

Joint Application Development

It is defined as a structured approach in which users, managers, and analysts work together for several days in a series of intensive meetings to specify or review system requirements. The important feature of JAD is **joint requirements planning**, which is a process whereby highly structured group meetings are conducted to analyze problems and define requirements.

The typical participants in a JAD are listed below:

JAD session leader:

The JAD leader organizes and runs the JAD. This person is trained in group management and facilitation as well as system analysis. The JAD leader sets the agenda and sees that it is met. The JAD leader remains neutral on issues and does not contribute ideas or opinions but rather concentrates on keeping the group on the agenda, resolving conflicts and disagreements, and soliciting all ideas.

(1) Users:

The key users of the system under consideration are vital participants in a JAD. They are the only ones who have a clear understanding of what it means to use the system on a daily basis.

(2) Managers:

The role of managers during JAD is to approve project objectives, establish project priorities, approve schedules and costs, and approve identified training needs and implementation plans.

(3) Sponsors:

A JAD must be sponsored by someone at a relatively high level in the company, i.e., the person from top management. If the sponsor attends any session, it is usually at the very beginning or at the end.

(4) Systems Analysts:

Members of the systems analysis team attend the JAD session although their actual participation may be limited. Analysts are there to learn from customers and managers, but not to run or dominate the process.

(5) Scribe:

The scribe takes down the notes during the JAD sessions. This is usually done on a personal computer or a laptop. Notes may be taken using a word processor. Diagrams may directly be entered into a CASE tool.

(6) IS staff:

IS staff like systems analysts, other IS staff such as programmers, database analysts, IS planners, and data centre personnel may attend to learn from the discussions and possibly to contribute their ideas on the technical feasibility of proposed ideas or on technical limitations of current systems.

The following are the various benefits of Joint Application Development:

- Actively involves users and management in project development.
- Reduces the amount of time required to develop a system.
- Incorporates prototyping as a means for confirming requirements and obtaining design approvals.

Chapter - 2

Documenting and evaluating the system

CONCEPTS AND PROCESS OF DOCUMENTATION

Documentation may be defined as the process of communicating about the system. The person who is responsible for this communication is called **documenter**. It may be noted that the documenter is not responsible for the accuracy of the information, and his job is just to communicate or transfer the information.

The ISO standard **ISO/IEC 12207:1995** describes documentation *"as a supporting activity to record information produced by a system development life cycle process."*

Why documentation?

Documentation is needed because it is:

- a means for transfer of knowledge and details about description of the system;
- to communicate among different teams of the software project;
- to help corporate audits and other requirements of the organization;
- to meet regulatory demand;
- needed for IT infrastructure management and maintenance; and
- needed for migration to a new software platform.

- Document communicates the details about the system targeted at different audience. It explains the system. The ever-increasing complexity of information system requires emphasis on well-established system of documentation. Every information system should be delivered along with an accurate and understandable document to those who will use the software.

Traditionally, documentation was done after the development of the software is completed. However, as the software development process is becoming complex and involved, documentation has become an integral part of each system development process. Documentation is now carried out at every stage as a part of development process. We will also discuss how documentation affects quality of the software later in this section.

When the process of documentation is undertaken as a separate process, it requires planning in its own right. The figure below shows how at the development process, documentation is done alongside each step. Design and development activities of software depend on a certain base document. Documentation is to be carried out before actually implementing the design. In such a case, any flaw in design identified can be changed in the document thereby saving cost and time during implementation. If documentation is being developed for an existing software, then documentation is done alongside the software development process.

The Process of Documentation

The following are various steps involved in the process of documentation:

Collection of source material:
The very first step of any documentation process is to acquire the required source material for preparation of document. The material is collected including specifications, formats, screen layouts and report layouts. A copy of the operational software is helpful for preparing the documentation for user.

Documentation Plan:
The documenter is responsible for preparation of a documentation plan, which specifies the details of the work to be carried out to prepare the document. It also defines the target audience.

Review of Plan:

The plan as set out in the process above is reviewed to see that material acquired is correct and complete.

Creation of Document:

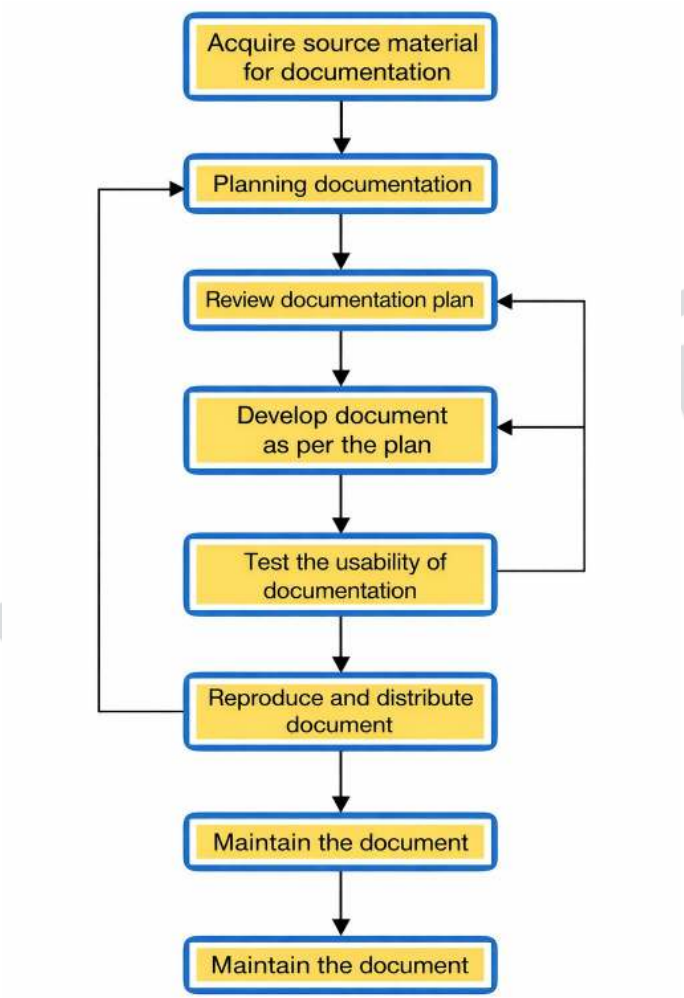
The document is prepared with the help of document generator.

Testing of Document:

The document created is tested for usability as required by the target audience.

Maintain Document:

Once the document is created and distributed, it must be kept up to date with new version of the software product. It must be ensured that the latest document is available to the user of the software.


Acquire source material for documentation
Planning documentation
Review documentation plan
Develop document as per the plan
Test the usability of documentation
Reproduce and distribute document
Maintain the document
Maintain the document

TYPES OF DOCUMENTATION

Any software project is associated with a large number of documents depending on the complexity of the project. Documentation that are associated with system development has a number of requirements. They are used by different types of audience in different ways as follows:

- They act as a means of communication between the members of development team.
- Documents are used by maintenance engineer.
- Documents are used by the user for operation of the software.
- Documents are used by system administrator to administer the system.

System Requirements Specification

System Requirements Specification is a set of complete and precisely stated properties along with the constraints of the system that the software must satisfy. A well-designed software requirements specification establishes boundaries and solutions of system to develop useful software. All tasks, however minute, should not be underestimated and must form part of the documentation.

Requirements of SRS:

The SRS should specify only the external system behaviour and not the internal details. It also specifies any constraints imposed on implementation. A good SRS is flexible to change and acts as a reference tool for system developer, administrator and maintainer.

Characteristics of a System Requirements Specification (SRS)

1. **All the requirements must be stated unambiguously.** Every requirement stated has only one interpretation. Every characteristic of the final product must be described using a single and unique term.
2. **It should be complete.** The definition should include all functions and constraints intended by the system user. In addition to requirements on the system as specified by the user, it must conform to standard that applies to it.
3. **The requirements should be realistic and achievable with current technology.** There is no point in specifying requirements which are unrealizable using existing hardware and software technology. It may be acceptable to anticipate some hardware developments, but developments in software technology are much less predictable.

4. It must be verifiable and consistent.

The requirements should be shown to be consistent and verifiable. The requirements are verified by system tester during system testing. So, all the requirements stated must be verifiable to know conformity to the requirements. No requirement should conflict with any other requirement.

5. **It should be modifiable.**

The structure and style of the SRS are such that any necessary changes to the requirements can be made easily, completely and consistently.

6. It should be traceable to other requirements and related documents.

The origin of each requirement must be clear. The SRS should facilitate the referencing of each requirement for future development or enhancement of documentation. Each requirement must refer to its source in previous documents.

7. SRS should not only address the explicit requirement but also implicit requirements that may come up during the maintenance phase of the software. It must be usable during operation and maintenance phase. The SRS must address the needs of the operation and maintenance phase, including the eventual replacement of the software.

Rules for Specifying Software Requirements : The following are the rules for specifying software requirements:

- Apply and use an industry standard to ensure that standard formats are used to describe the requirements. Completeness and consistency between various documents must be ensured.
- Use standard models to specify functional relationships, data flow between the systems and sub-systems, and data structure to express complete requirements.
- Limit the structure of paragraphs to a list of individual sentences to increase the traceability and modifiability of each requirement and to increase the ability to check for completeness. It helps in modifying the document when required.
- Phrase each sentence to a simple sentence. This is to increase the verifiability of each requirement stated in the document.

Structure of a Typical SRS Document

1. Introduction

- System reference and business objectives of the document.
- Goals and objectives of the software, describing it in the context of the computer-based system.
- The scope of the document.

2. Informative description about the system

- Information flow representation.
- Information content and structure representation.
- Description of sub-systems and system interface.
- A detailed description of the problems that the software must solve.
- Details of information flow, content, and structure are documented.
- Hardware, software, and user interfaces are described for external system.

3. Functional Description of the system

- Functional description.
- Restrictions/limitations.
- Performance requirements.
- Design constraints.
- Diagrams to represent the overall structure of the software graphically.

4. Test and validation criteria

- Performance limitation, if any.
- Expected software response.
- It is essential that time and attention be given to this section.

5. Glossary: Definitions of all technical or software-specific terms used in the document.

6. Bibliography : List and reference of all documents that relate to the software.

7. Appendix : Supplementary information to the specification.

System Design Specification

The system design specification or software design specification as referred to has a primary audience, the system implementer or coder. It is also an important source of information for the system verification and testing. The system design specification gives a complete understanding of the details of each component of the system, and its associated algorithms, etc.

The system design specification documents all as to how the requirements of the system are to be implemented. It consists of the final steps of describing the system in detail before the coding starts.

The system design specification is developed in a two stage process. In the first step, design specification generally describes the overall architecture of the system at a higher level. The second step provides the technical details of low-level design, which will guide the implementer. It describes exactly what the software must perform to meet the requirements of the system.

Tools for describing design

Various tools are used to describe the higher level and lower level aspects of system design. The following are some of the tools that can be used in the System Design Specification to describe various aspects of the design.

1) Data dictionary

Definition of all of the data and control elements used in the software product or subsystem. Complete definition of each data item and its synonyms are included in the data dictionary. A data dictionary may consist of description of data elements and definitions of tables.

Description of data element

- Name and aliases of data item (its formal name).
- Uses (which processes or modules use the data item; how and when the data item is used).
- Format (standard format for representing the data item).
- Additional information such as default values, initial value(s), limitations and constraints that are associated with the data elements.

Table definitions

- Table name and aliases.
- Table owner or database name.
- Key order for all the tables, possible keys including primary key and foreign key.
- Information about indexes that exist on the table.

Database schema

Database schema is a graphical presentation of the whole database. Data retrieval is made possible by connecting various tables through keys. Schema can be viewed as a logical unit from programmer's point of view.

E–R model

Entity-relationship model is database analysis and design tool. It lists real-life application entities and defines the relationship between real-life entities that are to be mapped to database. E–R model forms basis for database design.

Security model:

The database security model associates users, groups of users, applications with database access rights.

Trade-off matrix:

A matrix that is used to describe decision criteria and relative importance of each decision criterion. This allows comparison of each alternative in a quantifiable term.

Decision table:

A decision table shows the way the system handles input conditions and subsequent actions on the event. A decision table is composed of rows and columns, separated into four separate quadrants.

- Input Conditions | Condition Alternatives
- Actions | Subsequent action Entries

Timing diagram:

Describes the timing relationships among various functions and behaviours of each component. They are used to explore the behaviours of one or more objects throughout a given period of time. This diagram is specifically useful to describe logic circuits.

State machine diagram:

State machine diagrams are good at exploring the detailed transitions between states as the result of events. A state machine diagram is shown in Figure 4.2.

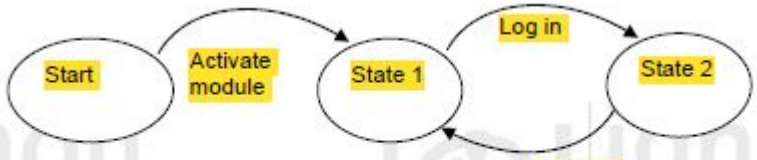


Figure 4.2 : A state machine diagram

Object Interaction Diagram

It illustrates the interaction between various objects of an object-oriented system. Please refer to figure 4.3. This diagram consists of directed lines between clients and servers. Each box contains the name of the object. This diagram also shows conditions for messages to be sent to other objects. The vertical distance is used to show the life of an object.

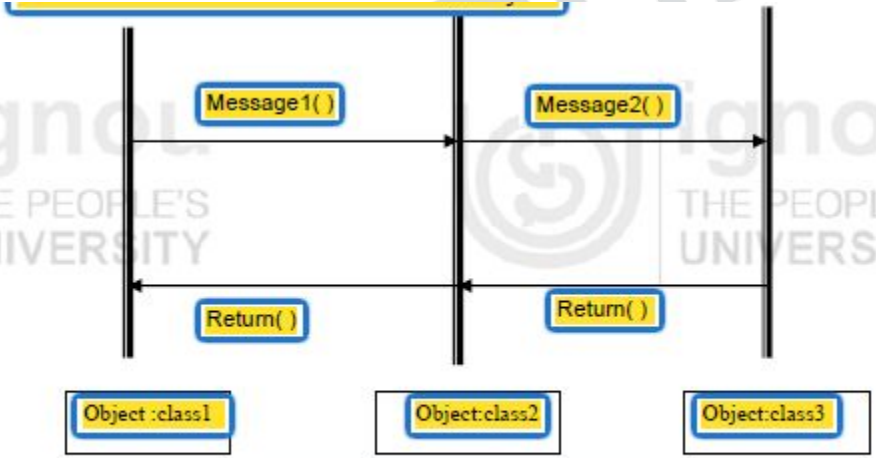


Figure 4.3: An object interaction diagram

Flow Chart

A Flow chart shows the flow of processing control as the program executes. Please refer to figure 4.4.

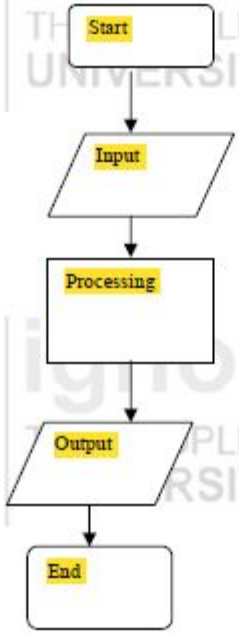


Figure 4.4 : Flow chart

Inheritance Diagram

It is a design diagram work product that primarily documents the inheritance relationships between classes and interfaces in object-oriented modeling. The standard notation consists of one box for each class. The boxes are arranged in a hierarchical tree according to their inheritance characteristics. Each class box includes the class name, its attributes, and its operations. Figure 4.5 shows a typical class diagram.

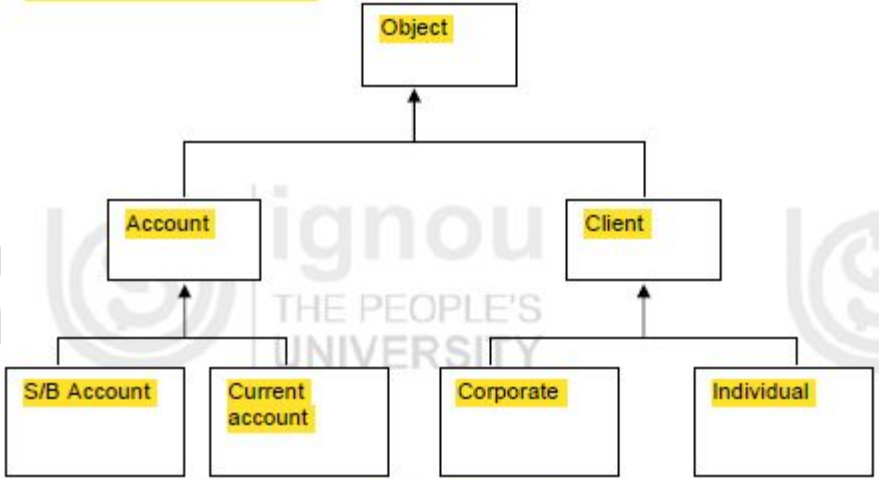


Figure 4.5: A typical inheritance diagram

Aggregation Diagram

The E–R model cannot express relationships among relationships. An aggregation diagram shows relationships among objects. When a class is formed as a collection of other classes, it is called an aggregation relationship.

Each module will be represented by its name. The relationship will be indicated by a directed line from container to contained. The directed line is labelled with the cardinality of the relationship. It describes “has a” relationship. Figure 4.6 shows an aggregation between two classes (circle has a shape).

Figure 4.6 : Aggregation

Structure Chart

A structure chart is a tree of sub-routines in a program (Refer to figure 4.7). It indicates the interconnections among the sub-routines. The sub-routines should be labelled with the same name used in the pseudo code.

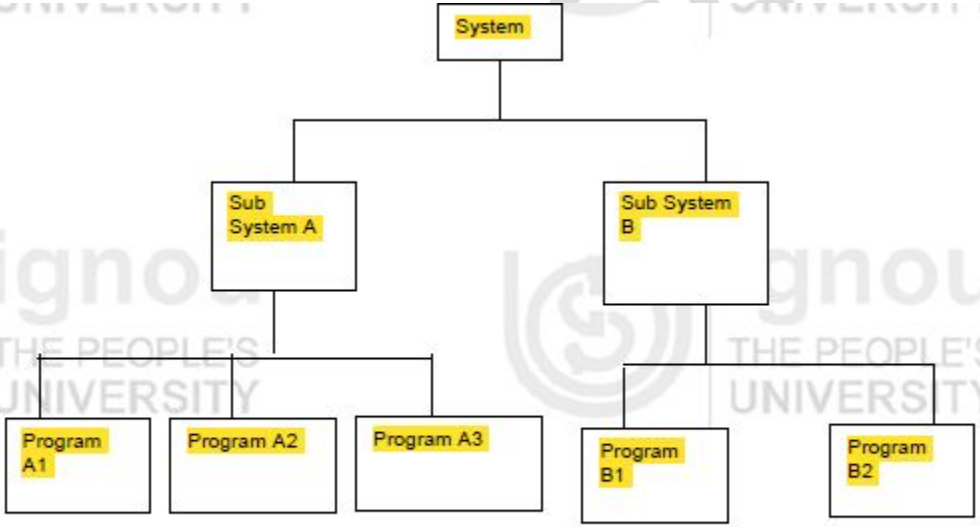


Figure 4.7 : A structures chart

Pseudocode

Pseudocode is a kind of structured English for describing algorithms in an easily readable and modular form. It allows the designer to focus on the logic of the algorithm without being distracted by details of language syntax in which the code is going to be written. The pseudocode needs to be complete. It describes the entire logic of the algorithm so that implementation becomes a routine mechanical task of translating line by line into source code of the target language. Thus, it must include flow of control.

This helps in describing how the system will solve the given problem. “Pseudocode” does not refer to a precise form of expression. It refers to the simplification of Standard English. Pseudocode must use a restricted subset of English in such a way that it resembles a good high-level programming language.

Figure 4.8 shows an example of pseudocode.

```
IF HoursWorked > MaxWorkHour THEN
    Display overtime message
ELSE
    Display regular time message
ENDIF
```

Contents of a Typical System Design Specification Document

1. Introduction

1.1 Purpose and scope of this document:

Full description of the main objectives and scope of the SDS document is specified.

1.2 Definitions, acronyms, abbreviations and references:

Definitions and abbreviations used are narrated in alphabetic order. This section will include technical books and documents related to design issues. It must refer to the SRS as one of the reference books.

2. System architecture description

2.1 Overview of modules, components of the system and sub-systems

2.2 Structure and relationships

- Interrelationships and dependencies among various components are described.
- Here, the use of structure charts can be useful.

3. Detailed description of components

- Name of the component
- Purpose and function
- Sub-routine and constituents of the component
- Dependencies, processing logic including pseudocode
- Data elements used in the component

4. Appendices

Test Design Document

During system development, this document provides the information needed for adequate testing. It also lists approaches, procedures and standards to ensure that a quality product that meets the requirement of the user is produced. This document is generally supplemented by documents like schedules, assignments and results. A record of the final result of the testing should be kept externally.

This document provides valuable input for the maintenance phase.

The following IEEE standards describe the standard practices on software test and documentation:

1. **829-1998** IEEE Standard for Software Test Documentation
2. **1008-1987 (R1993)** IEEE Standard for Software Unit Testing
3. **1012-1998** IEEE Standard for Software Verification and Validation

The following is the typical content of Test Design Document:

1. Introduction

Purpose

The purpose of this document and its intended audience are clearly stated.

Scope

Give an overview of testing process and major phases of the testing process. Specify what is not covered in the scope of the testing such as supporting or not third-party software.

Glossary

It gives definition of the technical terms used in this document.

References

Any references to other external documents stated in this document including references to related project documents. They usually refer the System Requirement Specification and the System Design Specification documents.

Overview of Document

Describe the contents and organization of the document.

2. Test Plan

A test plan is a document that describes the scope, approach, resources and schedule of intended testing activities. It identifies test items, the features to be tested, the testing tasks, and the person who will do each task, and any risks that require contingency planning.

2.1 Schedules and Resources

An overview of the testing schedule in phases along with resources required for testing is specified.

2.2 Recording of Tests

Specify the format to be used to record test results. It should very specifically name the item to be tested, the person who did the testing, reference of the test process/data and the results expected by the test, the date tested. If a test fails, the person with the responsibility to correct and retest is also documented. The filled-out format would be kept with the specific testing schedule. A database could be used to keep track of testing.

2.3 Reporting Test Results

The summary of what has been tested successfully and the errors that still exist which are to be rectified is specified.

3. Verification Testing

3.1 Unit Testing

For each unit/component, there must be a test which will enable tester to know about the accurate functioning of that unit.

3.2 **Integration Testing**

Integration test is done on modules or sub-systems.

4. Validation Testing

4.1 System Testing

This is the top level of integration testing. At this level, requirements are validated as described in the SRS.

4.2 Acceptance and Beta Testing

List test plans for acceptance testing or beta testing. During this test, real data is used for testing by the development team (acceptance testing/alpha testing) or the customer (beta testing). It describes how the results of such testing will be reported back and handled by the developers.

User Manual

This document is complete at the end of the software development process.

Different Types of User Documentation

Users of the system are not of the same category and their requirements vary widely. In order to cater to the need of different class of user, different types of user documentation are required. The following are various categories of manuals:

- **Introductory manual:** How to get started with the system?
- **Functional description:** Describes functionality of the system.
- **Reference manual:** Details about the system facility.
- **System administrator guide:** How to operate and maintain the system?
- **Installation document:** How to install the system?

The following is the typical content of User Manual

1. Introduction

1.1 Purpose

The purpose of this document and its intended audience is stated. If there is more than one intended audience, provide information in this section and direct the reader to the correct section(s) for his/her interest.

1.2 Scope of Project

Overview the product. Explain who could use the product. Overview the services that it provides. Describe limitation of the system. Describe any restrictions on using or copying the software and any warranties or contractual obligations or disclaimers.

1.3 Glossary

Define the technical terms used in this document. Do not assume that the reader is expert.

1.4 References

References to other documents cited anywhere in this document including references to related project documents. This is usually the only bibliography in the document.

1.5 Overview of Document

The contents and organization of the rest of this document are described.

2. Instructional Manual

This section should be divided in the manner that will make it user friendly.

2.1 System Usage

Provide examples of normal usage. Images of the screendumps are very useful to provide a look and feel of the product. Provide any necessary background information. On-line help system is very common for systems today. Information on how to use the on-line help system, how to access it may be provided here.

3. User Reference Manual

3.1 List of Services

Provides an alphabetical listing of services provided by the system with references to page numbers in this document where the concerned service is described.

3.2 Error Messages and Recovery

Provides an alphabetical list of all error messages generated by the system and how the user can recover from each of those errors.

4. Installation Information

Installation information is provided including the operating environment.

Maintenance Manual

The supplier/developer of the software is sometimes different from the software maintenance agency. In such cases, the criticality of maintenance manual assumes a bigger role. Maintenance manuals provide precise information to keep your product operating at peak performance. Similar to installation manuals, these documents may range from a single sheet to several hundreds of pages.

Different standards for documentation

The International Standards, **ISO/IEC 12207 – Software life cycle process**, describes documentation as one of the supporting parallel processes of software development process. It may be noted that this standard is not documentation standard but describes the process of documentation during the software development process. The following are other documentation standards:

1. ISO/IEC 18019: Guidelines for the design and preparation of user documentation for application software

This standard describes how to establish what information users need, how to determine the way in which that information should be presented to the users, and then how to prepare the information and make it available. It covers both on-line and printed documentation. It describes standard format and style to be adopted for documentation. It gives principles and recommended practices for documentation.

2. ISO/IEC 15910: Software user documentation process

This standard specifies the minimum process for creating user documentation for software that has a user interface, including printed documentation (e.g., user manuals), on-line documentation, help text, and on-line documentation systems.

3. IEEE 1063: Software Documentation

It provides minimum requirement for structure, information content and format for user documentation. It does not describe the process to be adopted for documentation. It is applicable for both printed and on-line documentation.

Components of software user documentation as described in IEEE 1063

1. Identification data (e.g., Title Page)
2. Table of contents
3. List of illustrations
4. Introduction
5. Information for use of the documentation such as description of software etc.
6. Concept of operations
7. Procedures
8. Information on software commands
9. Error messages and problem resolution
10. Glossary (to make the reader acquainted with unfamiliar terms)
11. Related information sources
12. Navigational features
13. Index 14.Search capability (for electronic document)

Documentation involves recording of information generated during the process of software development life cycle. Documentation process involves planning, designing, developing, distributing and maintaining documents.

During planning phase, documents to be produced during the process of software development are identified. For each document, following items are addressed:

- Name of the document
- Purpose
- Target audience
- Process to develop, review, produce, design and maintain

The following form part of activities related to documentation of development phase:

- All documents to be designed in accordance with applicable documentation standards for proper formats, content description, page numbers, figure/table numbers.
- Source and accuracy of input data for document should be confirmed.
- Use of tools for automated document generation.
- The prepared document should be of proper format. Technical content and style should be in accordance to documentation standards.

Production of the document should be carried out as per the drawn plan. Production may be in either printed form or electronic form. Master copy of the document is to be retained for future reference.

Maintenance: As the software changes, the relevant documents are required to be modified. Documents must reflect all such changes accordingly.

DOCUMENTATION AND QUALITY OF SOFTWARE

Inaccurate, incomplete, out of date, or missing documentation is a major contributor to poor software quality. That is why documentation and document control has been given due importance in ISO 9000 standards, SEI CMM software maturity model. In SEI CMM process model and assessment procedure, the goal is to improve the documentation process that has been designed. A maturity level and documentation process profile is generated from the responses to an assessment instrument.

One basic goal of software engineering is to produce the best possible working software along with the best possible supporting documentation. Empirical data show that software documentation products and processes are key components of software quality. Studies show that poor quality, out of date, or missing documentation are a major cause of errors in software development and maintenance. Although everyone agrees that documentation is important, not everyone fully realizes that documentation is a critical contributor to software quality.

Documentation developed during higher maturity levels produces higher quality software.

Chapter - 3

Data Modelling

DATA MODELING

It is a technique for organizing and documenting a system's data. Data modeling is sometimes called database modeling because a data model is eventually implemented as database. It is also sometimes called information modeling. The tool for data modeling is entity relationship diagram.

1) ER Diagram

It depicts data in terms of entities and relationships described by the data. Martin gives the following notations for the components of ERD.

a. Entities : An entity is any item about which the business needs to store data. An entity is a class of persons, places, objects, events or concepts about which we need to capture and store data. An entity instance is a single occurrence of an entity. The notation is given below:

Student

Student is the name of entity.

b. Attribute

An attribute is a descriptive property or characteristic of an entity. Synonyms include element, property and field. A compound attribute is one that actually consists of other attributes. It is also known as a composite attribute. An attribute “Address” is the example of compound attribute as shown in the following illustration.

c. Relationships

A relationship is a natural business association that exists between one or more entities. The relationship may represent an event that links the entities.

Important terms related to ER diagrams

Cardinality

Cardinality defines the minimum and maximum number of occurrences of one entity that may be related to a single occurrence of the other entity. Because all relationships are bidirectional, cardinality must be defined in both directions for every relationship. Figure 7.4 depicts various types of cardinality.

Degree

The degree of a relationship is the number of entities that participate in the relationship.

Recursive relationship

A relationship that exists between different instances of the same entity is called recursive relationship. Figure 7.3 depicts recursive relationship between the instances of the Course entity.

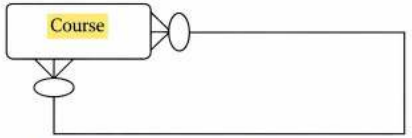


Figure 7.3: Example of Recursive relationship

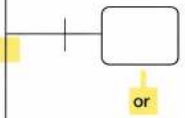
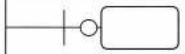
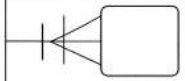
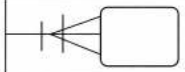
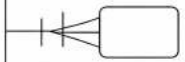
Cardinality Interpretation	Minimum Instances	Maximum Instances	Graphic Notation
Exactly one (One and only one)	1	1	
Zero or one	0	1	
One or more	1	Many (>1)	
Zero, one or more	0	Many (>1)	
More than one	>1	>1	

Figure 7.4: Different types of cardinality

Chapter - 4

Testing and Security Aspects

COMPUTER SYSTEM AND SECURITY ISSUES

Security is an important issue for modern IT systems. Even though technology provides immense possibilities to safeguard organizations' computing infrastructure, there have been security lapses and security breaches which have cost the organization heavily. System administrators and security administrators have spent sleepless nights to safeguard organization's data and computing infrastructure. One can think of organizations like airlines, railway and banks which are heavily dependent on computing infrastructure and unavailability of system for few hours can create havoc. Organizations can't afford to underestimate the security issues that can affect their business operations.

Degree of security = 1 − (No. of security failures / No. of attempt to breach security)

Implementation and Security of Systems & MIS

There may be security threats due to natural reasons such as Earthquakes, Cyclones etc. Sometimes, the threats are made by people. These may be due to riots, unrest, sabotage etc. Whenever, there are an attack, immediate reactive measures are to be taken. Also, one should study various controls to find out the people or reasons behind the attack. This can be done with the help of transaction logs etc. These attacks basically become possible due to several drawbacks in the information system such as lack of proper implementation of security protocols. Such things are exploited by people who plan attacks. The entire situation surrounding attacks is depicted in Fig. 12.1.

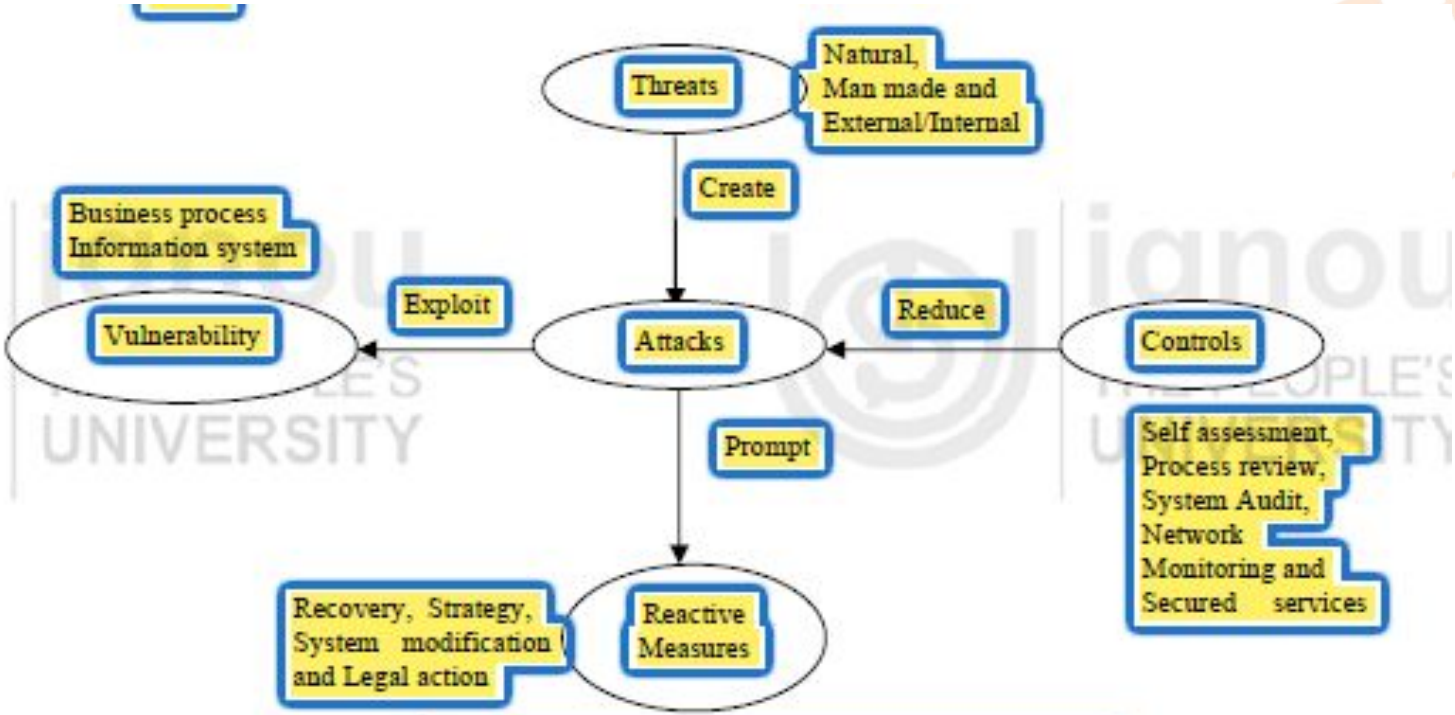


Figure 12.1: Information Security Architecture.

12.5.1 Analysis of Threats and Risks

The security of any system should be commensurate with the risk involved. Threat and risk assessment involves identification of applicable threats to IS infrastructure, recognition of vulnerability and probable loss calculation. In this context, it is necessary to identify the source of threat.

Historically, an organization's computer systems were centrally located and the management of issues related to it were responsibility of the computer center staff and as such security related issues were also the responsibility of computer center staff whose focus were to make available the application on the centrally located computer as required. In comparison, today's computing infrastructure are far more diverse and complex to manage. Business information is dispersed.

-

The source of threats can be either external or internal. Historically virus has been the major potential external security threat but as organizations are diversifying their activity over multiple locations and with evolution of new technology it is difficult to perceive when an unauthorized intruder may try to hack upon organization's vital information and cause damage. Internal security threats are more common although the integrity of employee is checked before being inducted into the organization. Employee of an organization can pose serious threats to information security as they are closely associated with the system and know the vulnerabilities that can be targeted.

Risk Analysis

The common questions asked in evaluating the risks are given below.

- Are the risks such as fire, earthquakes and the scope of their effects on the information system been made clear?

- Has the loss the organization would suffer from a halt or the like of the information system been analyzed?
- Is the time permissible for recovery of operation and the order of priority of recovery been determined?

Security policy underlines an organization's holistic approach to security issues. Organizations must possess a security policy in writing, which should address the following issues:

- **Authentication:** To see that the person is bona fide user of the resources
- **Authorization:** Privileges of the user or who can do what?
- **Information integrity:** Is it possible that the end user can modify the information?
- **Detection:** Once the problem is identified, how it is handled and managed.

Risk Assessment and Management

A thorough and proactive risk assessment is the first step in establishing a sound security system. This is the ongoing process of evaluating threats and vulnerabilities, and establishing an appropriate risk management program to mitigate potential monetary losses and harm to an institution's reputation. Threats have the potential to harm an institution, while vulnerabilities are weaknesses that can be exploited.

There are different approaches followed by organizations to analyze risks. However, ultimately all the methods boil down to two types of approaches: **quantitative** and **qualitative**.

Quantitative Risk Analysis

This approach, although difficult to implement, gives an idea about the amount of risk involved with the event. This basically employs two fundamental elements i.e., the probability of occurrence of the loss making event and probability of occurrence of the event.

Estimated Loss = Potential loss due to the event × Probability

It is therefore possible to rank the events in order of estimated loss. But the problem associated with the quantitative approach is estimating the probability of occurrence of the event, also some cases the events are interrelated making probability calculation even more difficult. Notwithstanding above difficulty, many organizations have adopted and implemented this approach successfully.

Qualitative Risk Analysis

The extent of the information security program should commensurate with the degree of risk associated with the institution's systems, networks, and information assets. For example, compared to an information-only Web site, institutions offering transactional Internet banking activities are exposed to greater risks. Further, real-time funds transfer generally pose greater risks than delayed or batch-processed transactions because the items are processed immediately. The extent to which an institution contracts with third-party vendors will also affect the way the risk assessment has to be done.

Performing the Risk Assessment and Determining Vulnerabilities : Performing a sound risk assessment is critical to establishing an effective information security program. The risk assessment provides a framework for establishing policy guidelines and identifying the risk assessment tools and practices that may be appropriate for an institution. Banks still should have a written information security policy, sound security policy guidelines, and well-designed system architecture, as …

as well as provide for physical security, employee education, and testing, as part of an effective program.

When institutions contract with third-party providers for information system services, they should have a concrete opinion about third-party provider's quality of work and loyalty to the clients. At the minimum, the security-related clauses of a written contract should define the responsibilities of both parties with respect to data confidentiality, system security, and notification procedures in the event of data or system compromise. The institution needs to conduct a comprehensive analysis of the provider's security program, including how the provider uses available risk assessment tools and practices. Institutions should obtain copies of independent penetration tests and risk assessment reports against the provider's system.

When assessing information security products, management should be aware that many products offer a combination of risk assessment features and can cover single or multiple operating systems. Several organizations provide independent assessments and certifications of the adequacy of computer security products (e.g., firewalls). While the underlying product may be certified, management should realize that the manner in which the products are configured and ultimately used is an integral part of the certification's effectiveness. In relying on the certification, banks should understand the certification process used by the organization certifying the security product. Other examples of items to consider in the risk assessment process include:

- Identifying mission-critical information systems, and determining the effectiveness of current information security programs. For example, vulnerability might involve critical systems that are not reasonably isolated from the Internet and external access via modem, up-to-date inventory lists of hardware, software, as well as network topologies, important in this process.

- Assessing the importance and business sensitivity of information for the likelihood of outside attacks/hacking and internal misuse of information. For example, organizational data could be harmed if information resource data (e.g., confidential personal information) were made public. The assessment process should identify systems that review the appropriateness of access controls and other security policy settings.
- Assessing the risks posed by service provider or business partner through electronic connections with internal IT infrastructure. The outsider may have poor access controls that could potentially lead to an indirect compromise of the organization's security system.
- Determining legal implications of security breaks and containing liability concerns associated with any of the above factor. For example, if hackers successfully access a bank's system and withdraw money fraudulently, the bank will be liable for damage incurred to the account holder.

Potential Threats

- **Denial of service (DoS):** which can be described as any action that prevents a system from normal operation. It may be the unauthorized destruction, modification, or delay of service. DoS is common where the number of requests outnumber the maximum number of connections possible. Under such circumstances, legitimate users have to wait for large amount of time for response to their request.
- **Internet Protocol (IP) spoofing:** which allows an intruder via the Internet/intranet to effectively impersonate a local system's IP address in an attempt to gain access to the system. The system has easy maintenance incoming connection as originating from a trusted host.
- **A Trojan horse program:** Generally performs unintended destructive functions that may include destroying data, collecting invalid or falsifying data. Trojan horses can be attached to e-mails.

- **Viruses** are computer programs that may be embedded in other programs and have the capability of self-replication. Once active, they may result in either a non-destructive or destructive invalid outcomes in the host computer. The virus program may also move into multiple platforms, data files, or devices on a system and spread through multiple systems in a network or through emails to other systems.

Recovering from Disasters

Natural and man-made disasters are inevitable. Earthquake, floods, fire and terrorist attack can severely damage organizations computing infrastructure. The disaster recovery plan is a document containing procedures, extended backup operations, and recovery should a computer installation experience a partial or total loss of computing resources or physical facilities (or access to such facilities).

The primary objective of this plan, used in conjunction with the contingency plans, is to provide reasonable assurance that a computing installation can recover from disasters, continue to process critical applications in a degraded mode, and return to a normal mode of operation within a reasonable time. A key part of disaster recovery planning is to provide for processing at an alternative site during the time that the original facility is unavailable.

Contingency and emergency plans establish recovery procedures that address specific threats. These plans help prevent minor incidents from escalating into disasters. For example, a contingency plan might provide a set of procedures that define the condition and response required to return a computing capability to normal operation. An emergency plan might be a specific procedure for shutting down equipment in the event of a fire or evacuating a facility in the event of an earthquake.

During a disaster, normal operating procedures may be significantly altered. Both personnel and systems will be expected to function under conditions that are not expected under normal day-to-day operations. Security remains a requirement but techniques to apply it are altered to fit the contingency situation.

In-house Backup

This level is the minimum acceptable and is mandatory for all installations and applications' systems. Define in detail all in-house backup procedures, the techniques used, files copied, frequency, etc.

Alternate Storage Area

This level of protection is necessary for mission critical components. It consists of off-site storage of at least one copy of all IS files and databases, programs, and procedures necessary to operate the high priority application systems, either at the installation or at an alternate site of operation (including copies of contingency plans and related materials).

The alternate storage area should be located in an area reasonably accessible to the installation, but not subject to the same degree of major threat as the site. It is recommended that, as a rule of thumb, the alternate storage area be no closer than one mile from the site. However, the distance may vary from location to location.

The Disaster Recovery Toolkit

The Disaster Recovery Toolkit is a highly valuable collection of items and documents to assist in ensuring business continuity in the face of serious incident or disaster. Many organizations use these documents as a checklist and add elements specific to their need. Although they vary from organization to organization, they generally comprise the following:

- A contingency audit questionnaire
- A dependency analysis document – questions and guidance
- A Business Impact Analysis questionnaire
- An audit questionnaire for disaster recovery or business continuity plan
- A checklist, action list and framework for disaster recovery

The toolkit is designed to help review the full spectrum of business continuity and disaster recovery issues.

Planning the Contingencies

Every business entity can and do experience events which can prevent it from normal function. The factors can range from natural events like flood, fire, earthquake or man-made events like unauthorized access, serious computer malfunction or various information security accidents.

The very first step for contingency planning is to identify the contingency events to be covered and the appropriate actions for each contingency event; usually refer to varying degrees of loss across six major asset categories: Data, Software, Communications, Hardware, Personnel, and Facility. The cause of the loss is dealt with in the Risk assessment, the primary concern in contingency plan is the degree of loss, impact on the mission and techniques for coping.

Contingency management tools address basic issues such as asset identification, location, value, alternative, replacement, and intangible costs; and most importantly, how long can the organization function without the asset. Since no asset is impervious to loss, the prudent leader will ensure that mechanisms are in place for a secure & rapid recovery. Our intent is to help managers break the cycle from normality to panic with crisis management.

Contingency Events

Loss of Data:

To identify key data and the type or degree of loss/damage that would be required for necessary recovery action. It can be done as follows:

- Identify appropriate recovery plan and procedure procedures (Example: in-house backups, etc.).
- The location of the required recovery files.

- To identify procedures for recovery of the files indicated above and include them in the contingency plan.

Loss of Software:

To identify key software and the degree of criticality for necessary recovery action. It can be done as follows:

- Identify the type of software (commercial / in-house developed).
- Identify the location where backup copies are maintained.
- In case of an emergency procurement process, the authorized person for it and any alternate source from where operational copies can be obtained.

Loss of Communications:

To identify voice as well as data communications loss for necessary contingency plan and recovery action. It can be done as follows:

- Identify alternate communication facility available such as radios links or mobile phones for interim measures.
- Whether there is any service agreement in place with any party to deal with contingency issues.
- To estimate recovery time.

Loss of Hardware

Inventory of required hardware must be maintained. For each hardware component, the loss which would require implementation of the contingency plan has to be found. It can be done as follows:

- Identify the hardware component and what functionalities it supports.
- Identify any alternate piece of equipment that may be used as a substitute to the equipment and it’s degree of compatibility with the existing software.
- Whether the equipment is repairable, and if so, is there maintenance agreement in place to accomplish the repairs. What is the response time to repair the equipment?
- The estimated cost and time for procurement of replacement hardware in case it is not repairable.
- Whether there are any emergency procurement procedures for key items?

Loss of Personnel

Loss of personnel can result from employee leaving the organization, illness, death, family emergency and number of other events. The following steps can be taken to minimize this type of loss:

- To identify key personnel in the organization and what their involvement/impact on major systems/programs/components.
- To identify substitutes for each personnel to handle such situation.
- Whether there are written procedures for every important function accomplished by the key personnel. Whether the substitutes use the same procedures periodically and do the assigned tasks.

- If alternates are not available within the organization, replacements must be obtained from outside sources. It must be ensured that there are sufficient procedures in place and establish a training/orientation program to assign them the desired work.

Loss of Facility

The loss of facility in general is due to some catastrophic natural action such as fire, flood, storm, earthquake, etc. However, a facility may become non-functional temporarily due to failure of power, or any other events that could render the facility non-functional.

- If the facility is to be out of operation beyond the maximum tolerable time, identify the procedures that are necessary for moving to the alternate facility.
- To identify all necessary hardware, software, data, and personnel required for normal functioning at the alternative location.
- To notify the alternate location to all concerned.

Recovery Procedures

Any recovery procedure generally consists of following broad steps:

Preparing contingency plan:

Involves people from all activities. The people should understand their role in the event of disaster and should be ready to react to the situation. Following are the major steps involved in contingency planning.

Develop the Plan:

The contingency plan is a detailed milestone to move the organization from a disrupted status to the status of normal operation. The role and responsibility of each employee and service provider are defined clearly in the event of disaster.

Testing the Plan:

Once the plan is ready, it should be subjected to rigorous testing and evaluation. The plan should be initially tested in a simulated environment. Persons who would actually be involved in the event of a real disaster should test the plan.

Maintaining the Plan:

Once the plan is created and tested it must be kept updated so that it remain relevant and applicable to changed business environment. The changes …

Viruses

Viruses are one of the major security threats to computer system. The first computer viruses were written in mid-eighties. The first virus written was a boot sector virus. Today, there are several tens of thousands of viruses.

Computer virus is nothing but a program that is loaded into your computer without your knowledge. This is only basic information. But, what makes people fear from virus is the disastrous impact on remaining programs in your machine to this program. The difference between a computer virus and other programs is that viruses are designed to self-replicate without the knowledge of the user. Computer viruses are called viruses because they share some of the traits of biological viruses. A computer virus passes from computer to computer like a biological virus passes from person to person. A computer virus must piggyback on top of some other program or document in order to get executed. Once it is running, it is then able to infect other programs or documents. Obviously, the analogy between computer and biological viruses seems superficial, but there are enough similarities as the name suggests.

Viruses cause out instructions for replication. The effect of virus can vary from annoying messages, to the disastrous consequences (for example, the CIH virus, which attempts to overwrite the FLASH BIOS, can cause irreparable damage to certain machines). Supervirally, it looks as if virus which can format hard disk is more damaging, but damage can be avoided by taking backups. Think of a virus which corrupt data by changing the numbers randomly on a spreadsheet application of changes up to 0–1%. This is certainly disastrous.

Viruses can be hidden in programs available on floppy disks or CDs, hidden in email attachments or in material downloaded from the web. If the virus has no obvious payload, a user with anti-virus software may not even be aware that the computer is infected.

A computer that has an active copy of a virus on its machine is considered infected. The way in which a virus becomes active depends on how the virus has been designed, e.g., macro viruses can become active if the user simply opens, closes or saves an infected document.

Prevention

The best way for users to protect themselves against viruses is to apply the following anti-virus measures:

- Make backups of all software (including operating systems). So, if a virus attack is been made, you can retrieve safe copies of your files and software.
- Inform all users that the risk of infection grows exponentially when people exchange floppy disks, download web material or open email attachments without caution.
- Have anti-virus (AV) software installed and updated regularly to detect, repair and disinfect viruses.
-

- Visit sites which give information on the Internet about latest virus, it’s behavior and assess their potential threat.
- In case of doubt about a suspicious item that anti-virus software does not recognize, contact your anti-virus team immediately for guidance.

Chapter - 5

Implementation of systems

IMPLEMENTATION OF SYSTEMS

Implementation of system involves developing working computer software from the design specification through coding by a team of programmers. Many times, the user requirements are either not built into the design specification or compromised to make the design simple and manageable. Implementation of the system is done by coding, testing and creating the necessary hardware and network environment, and imparting training to the end users. Of course, apart from Coding and Testing, the running implementation activities differ from project to project. This phase of the software development requires intensive user involvement.

Conducting System Tests

No system can be perfect. Testing is of vital importance as all information systems are designed by a team of Software Engineers and end users have little or no knowledge of system development. Testing is done to bridge the gap between the perceived outcomes desired by the user to that of systems analysts and programming team. The design specifications are requirements of the user and are translated to working software by the programmer. Hence, it is the ability of the programmer to code exactly as per the design specification that is to be judged by testing the software module.

The objective of any testing mechanism is to discover and fix bugs before the product is delivered to the customer. A good testing scheme has a high probability of discovering undiscovered errors. The objective of any good testing scheme is to find and fix bugs with minimum time and resources. Besides, bugs and errors in systems are tested for response time, volume of transactions that can be handled, stress under

which it can function, security and usability. For an Online Transaction Processing System, testing of the system for response time could be quite vital.

System testing assumes that all parts of the system are correct and error-free. Even though the system has been tested for individual components and modules, there is no guarantee that the system after integration will work as per the desired specification. System test involves a holistic approach for testing the working of the application in totality.

The following are various types of System Testing:

Recovery Testing:

Tests the ability of the system to recover from errors. Errors or any other processing faults must not cause overall system failure. The recovery time of the system after failure must be within a specified period and tolerance limit. System failures are forced during this phase of testing by introducing exceptions to see how the system responds to the case.

Security Testing:

System used for processing sensitive information are prone to high security risks. Individual often tries to access unauthorized data for various reasons. Threats could be external or internal. Hacking of passwords is a common problem. Individual can use software to generate random passwords to gain access of the system. Security testing takes care of these aspects of the system security.

Stress Testing:

Stress test is designed to test the system as to how the system behaves in an abnormal situation. The aim of the stress test is to find the limit of quantity or frequency of input after which the system fails. Stress test cases are designed which require maximum memory and other resources, in excess of what a normal situation demands.

Performance Testing:

Performance testing is specifically important to embedded and real-time systems. It checks the run time performance of the system. It is often coupled with stress testing.

Response Testing:

Testing of response time is of special importance in OLTP (online transaction processing systems like railway reservation system, points of sale, etc.). Testing is done to measure the response time. The same is compared with desired maximum response time.

Usability and Documentation Testing:

Testing is done to ensure that the usability and user friendliness of the software. Most often, systems are provided with on-line help, screen to help the end user. This also includes whether proper care had been taken to document the development stage of the project. User friendliness of the system is often compromised, which may lead to problems during implementation and maintenance of the system.

The following are the various activities involved during system testing:

Preparation of Test Plan:

The first step in system testing is to prepare a document called a Test Plan. Test plan is a document which outlines the aspect of the system to be tested. A workable Test Plan is prepared in accordance with the design specification such as:

- Expected output from the system;
- Criteria for evaluating the output;
- Nature and Volume of test data; and
- Procedure for using the test data.

Specifications of User Acceptance Test:

User is involved to prepare test cases. These can be derived from the test plan. Other parameters are test schedule, test duration and the person delegated to carry out the user acceptance test.

Preparation of Test Data for Testing:

Test data are often generated during testing of program. The test data must be representative of the live data to be actually used by the end users after installation. Care should be taken to select the nature and volume of data.

Although enough care is taken to test the system as per the documented specification, it is almost always a confusing regarding how the user will use the end product. In case there is one customer (a specific application designed for a specific user), a series of acceptance tests are carried out to validate all the user requirements. But this is not possible if the software is to be used by many customers (general purpose software like word processor, etc.). An alternate approach is application of Alpha and Beta testing techniques.

Alpha Testing:

Alpha testing is carried out by the customer at the developer's site. The customer uses the software and records the errors/bugs and usage problem. Alpha testing is carried out in a controlled environment.

Beta Testing:

Beta testing is carried out at one or more customer sites by the end users. It is live testing of the software product and not controlled by the developer. The customer tests the software using his/her own data and records and reports the bugs or problems in regular intervals to the developer.

Many organizations deploy specialized trained personnel for system testing. The problems of bugs uncovered in alpha and beta testing are fixed before the product is shipped and installed in customer's premises. Testing of complex software can be time consuming and frustrating also. The aim of system testing is to uncover every possible error that may come up at the user end. The role of Data Processing Auditor (EDP auditor)/Information System Auditor is quite involved during all stages of system development especially during testing. Auditors can provide useful independent inputs to minimize complications during maintenance.

Preparing Conversion Plan

A conversion plan is a document which spells out detailed requirements for a successful conversion from existing system to proposed system. The complexity of conversion is directly proportional to the complexity of the system in question. An important role of Systems Analyst is to see that the newly designed system is implemented to the set specification. Conversion is just one aspect of implementation, other being software maintenance and system review.

A proper conversion plan ensures that conversion from old system to new system is smooth without affecting the normal functioning operation. The conversion process can be tedious and disrupt normal functioning of system and also involves financial and human resources. A well-designed conversion plan facilitates a smooth switch over to the new system while keeping the cost and human involvement to the minimum.

A typical conversion plan is a document that consists of the following information:

- Guidelines regarding Conversion processes involved and the roles of end user.
- Planning conversion of files, creation of computer readable files.
- Types of conversion to be undertaken depending on the existing types of system. It could be from an existing manual system to a newly designed system or from an existing old computerized system to a newly designed enhanced system.
- Types of conversion may be parallel, phased or direct.
- Evaluation of hardware, software and related services.

Installing Database

Installation of database is nothing but creating computer readable files from the existing systems/documents. Each installation involves data. The new system is going to use data created either manually or data that has been obtained from the old system. If the current system is using computer readable data, it must be made error free and compatible for use in the new system. The data must be converted to the new format supported by the current technology on which the system is being developed.

Usually, there will be upward compatibility between various versions of software. The data conversion process can be tedious depending on the format supported by the old system. Special software are designed to facilitate the installation of database.

Training the End User

Training the user is one of the vital activities. The project team must make sure that the end users are trained to operate the new system. Many systems fail to get implemented or deliver the desired result because the end users are not trained.

Managers and the users must be trained on fundamentals of information technology in addition to knowing the operation of the new system. Training and support from the two crucial issues involving success of any information system. While training is imparted in a fixed schedule, support is an ongoing process. In support activity, the user is provided continual operational and technical support to carry out the work. Support materials are developed to facilitate this task. The goal of any training and support activity is to achieve highest possible productivity with lowest cost. Training may involve the following activities:

- Entering the data into the system; generating the required reports.
- Basic training of computer not specific to the application program like copying a file, starting and shutting down system, etc.
- Briefing about hardware and software concepts.
- Reporting non-compliance and bugs in the program; process of taking backup of daily work.

There is no exhaustive list of training requirement of the end user and can vary depending on the nature of application. The training must be scheduled in logical sequence depending on the pre-requisites for the next module of the training. A dependency chart could be useful for this purpose.

Training can be imparted in different ways:

- Computer-aided training
- Classroom tutorial
- Interactive training manual
- Resident technical expert
- One to one training
- External sources
- Information center / help desk

Many organizations have a well-developed automated support mechanism for end user support. To make a system success, following issues should be taken care of:

- Develop a satisfactory support base for providing support to the user;
- Obtain user participation and commitment;
- Institutionalize the system of training; and
- Insist on mandatory use of information system.

Regular training should become part of the organization's policy as information system changes as per the requirement of the organization and new features are added.

Preparing User Manual

All information systems are unique and different from one another. Documentation starts from the day one of system development lifecycle, but preparation of end user documentation is of specific importance as the end user documentation is the interface of system development and hence operational problems are bound to occur. Documentation of any information system is generally of two types: System Documentation and User Documentation. System documentation contains detailed information about system design specification, its internal structure and related technical details. The system documents are primarily for the programmer for maintenance purpose. The user documentation on the other hand is for the end user. The document should be structured and self-contained.

A user manual generally contains written as well as pictorial representation of the information system about its working and application. A well-designed user manual can reduce the overall cost of training and support. On-line help system with hyperlinks and context sensitive help systems are slowly replacing bulky and non-interactive documents.

The following are the components of a User Manual:

- Title and Version of software release
- Table of contents
- Salient features of the product
- Installation Guide and System requirements
- Getting started
- Frequently asked questions
- Sample scenario

- Glossary of terms used in the manual
- Known bugs in the applications

With changing technology, user documentation is often bundled to the information system. It is separate document. On-line documentations are being extensively utilized in user environment due to their convenience. Context sensitive helps are making the users' life easy by reducing the time to browse the bulky documents.

Converting to the New System

Actual conversion process involves equipments, personnel, data and financial resources. The process of conversion from the existing system (manual or computerized) to the newly developed system can be performed in several ways depending on the criticality of the system and other related issues.

Direct Conversion:

This is abrupt approach. The old system is shutdown and the new system starts. This kind of conversion although economical, the users are at the mercy of the new system, hence direct installation can be very risky. Some times due to procedural reasons where two systems can't be run parallel, this kind of conversion is the only option. When the new system fails, there is no way to start the old system as it has been shutdown. This kind of conversion plan is often the least preferred for critical business applications.

Pilot Conversion:

This is the middle path approach. Instead of converting all at once throughout the organization, this kind of pilot installation involves conversion at one selected location or system at a single pre-decided location. The location can be a branch office of the organization. Proper selection of the pilot site is important as it should be able to perform a true conversion process to test all functionalities of the new system. The advantage of the pilot conversion is that the potential risk in case of failure of the system is limited to a single location. Once the user is ascertained that the implementation of the system has been successful in a particular location, it is proposed to replicate the system in other locations. Although this kind of pilot conversion plan is beneficial for the user, it places a substantial burden on the implementation team as it has to maintain two systems in parallel.

Phased Conversion:

This is an incremental approach to switch over to the new system. Different sub-systems of the new system is used in conjunction until the whole new system is converted. This kind of approach for conversion limits the potential risk of failure of the new system. In a phased installation, as a sub-system is made functional, actual results are visible before the whole new system is made functional.

Parallel Conversion:

This is least risk prone. Under this kind of conversion, the old system is allowed to run alongside the new system until the management and the end user are satisfied with the result of the new system. This compares with the new system to test whether the functionalities covered by the old system are thoroughly covered in the new system by comparing the outputs. Errors and bugs identified with the new system are rectified for normal functioning of the organization as the new system is replaced and normal functions are resumed by the old system. Parallel conversion is costly as two systems are run in parallel, but results of only one system are used for business operations.

MAINTENANCE OF SYSTEMS

Once the information system is successfully installed and started showing results, the next issue is to maintain the system. System maintenance involves more than 80% of the total life of a software product; this shows the importance of maintenance. System maintenance is the task of monitoring, evaluating and modifying the information system to make necessary desirable changes during the total life cycle of the software. Organizational requirements as perceived during the analysis phase changes, the system has to accommodate all such changes to make the system current and useful for the organization. Maintenance of systems also takes care of the failure and shortcomings that arise during the operation of the information system by the end user.

During the implementation phase, one person from the system maintenance group is nominated to collect information from the users for maintenance. Maintenance activity involves collecting requests for changes, transforming these requests to changes, designing the changes to be incorporated and implementing the changes in the system.

Any maintenance activity comprises the following four key stages:

- **Help Desk:**
  The problem is received from the user through a formal change request. A preliminary analysis of the change request will be done, and if the problem is sensible, it is accepted.
- **Analysis:**
  Managerial and technical analyses of the problems are undertaken to investigate the cost factors and other alternative solutions. Feasibility analysis is done to assess the impact of the modification, to investigate alternative solutions, to assess short and long term costs, and to compute the benefit of making the change.

- **Implementation:**

  The chosen change/solution is implemented and tested by the maintenance team. All infected components are to be identified and brought into the scope of the change. Unit test, integration test, user-oriented and acceptance tests and regression tests are provided.

- **Release:**

  The changes are released to the customer, with a release note and appropriate documentation giving details of the changes.

Different Maintenance Activities

Once the system is fully implemented and starts operating, the maintenance phase begins. When the user starts operating the system, initial difficulty diminishes as the user learns to operate the system. The maintenance may include modification of system due to changes in business environment, government regulations, new business ventures and enhancement of functionalities.

The majority of Software Maintenance activity is concerned with evolution driven from user requested changes. A program that is used in a real world environment is necessarily must change or become progressively less useful in that environment. As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving the semantics and simplifying the structure.

-

- **Corrective Maintenance:** This type of maintenance is to rectify design, coding and implementation problems detected after the implementation of the system. This kind of problem generally surfaces immediately after the system is implemented. This type of problem needs immediate attention as it hampers the day-to-day work of the end user. Proper planning and interaction with the end user during system development process can minimize corrective maintenance. In spite of all these kinds of maintenance, these constitute more than 60% of total maintenance effort. Corrective maintenance is very much undesirable. It does not do any value addition to the software. Care should be taken to see that normal business operations are not disturbed because of it.

- **Adaptive Maintenance:**

  Changes are needed as a consequence of upgrade version or changes in operations system, hardware, or DBMS. Adaptive maintenance is required because business operates on a social environment and need of the organization changes as organization ventures into new areas, or as government regulatory policy changes, etc. Maintenance of the software to adapt to this kind of changes is called adaptive maintenance. Unlike corrective maintenance, this kind of maintenance activity value to the information system and affects a small part of the organization. This activity is not as urgent as corrective maintenance as these changes are gradual and allow sufficient time to the system group to make changes to the software.

- **Perfective Maintenance:**

This kind of maintenance activity involves adding new functionalities and features to the software to make it more versatile and user oriented. Some times, changes are made to improve performance of the software. In some sense, this maintenance can be thought of as a new development activity. This adds value to the information system and is required to stay ahead of the competition.

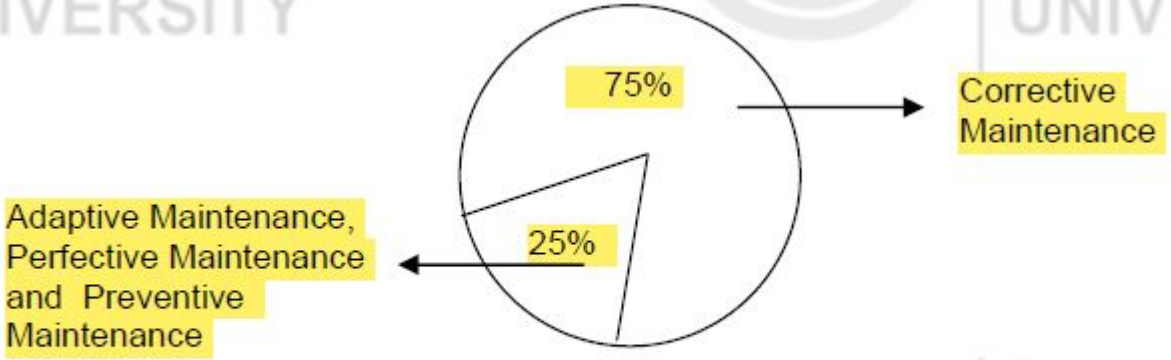


Figure 11.1: Comparative figures of maintenance effort.

- **Preventive Maintenance:**

  Changes are made to software to make it easily maintainable and to prevent any kind of system failure in future. This reduces the need of corrective maintenance. As corrective maintenance could lead to hamper normal functioning, preventive maintenance is done periodically to ensure that the probability of system failure is minimized. Preventive maintenance could increase the volume of transactions that can be handled by the system. Preventive maintenance is done when the system is least used or not used at all. This does not add value to the system, but certainly lowers the cost of corrective maintenance.

Issues Involved in Maintenance

The responsibility of the software development team and clients does not end once the product is released for implementation and installed. If software is not properly maintained, a well-documented and cleanly designed system can decay into a poorly documented and ill-maintained system. Additionally, the maintenance phase may get introduced during the activity of maintenance. In a network environment, a bug has ramifications beyond just poor performance or functionalities. A bug can open up avenue for a hostile intruder.

It is very important that the software should be easily maintainable. Factors like availability of source code, availability of system manuals, etc., are very important for maintainability. One of the most important issues is the cost factor for maintenance of software. There are a number of factors that influence the cost of maintenance. Maintenance activity may sometimes introduce new bugs while rectifying

The following are various factors which affect the ease of maintenance:

- **Volume of Defects:**
  The inherent errors / bugs that are found in the system after installation. Cost of maintenance increases with the increase in volume of defects.
- **Number of Customers:**
  More number of customers means more requests for changes in the system after installation.
- **Availability of System Documentation:**
  The quality and availability of system documentation is vital to carry out the maintenance. Poorly written system documentation increases the cost of maintenance. Most often, the programmers for development are different than the team of programmers for maintenance and the latter often finds it difficult to understand a program written by the former. Structured programming and program documentations are very useful in maintaining the system.

The following are various issues in Software Maintenance:

1. Organizational Issues

- Maintenance activity must align with organizational objectives.
- Most of the Software Maintenance activity is resource consuming and it has no clear quantifiable benefit for the organization.
- Outsourcing the job of Software Maintenance.

2. Process Issues

- Software Maintenance requires a number of additional activities not performed at development stage. Impact analysis and Regression tests on the software changes are crucial issues.

3. Technical Issues

How to construct software that is easy to comprehend is a major issue and the technology to do this is still not available. Still, the following are some guidelines for the same:

- Translate the problem into software terms to decide if it is viable or not.
- Determine the origin of the change request and suggest solutions.
- All solutions are investigated to determine that they are applied to all software components affected.
- Make a decision on the best implementation route or to make no change.
- Ripple effect propagation is a phenomenon by which changes made to a software component along the software life cycle have a tendency to be felt in other components.

Legacy System

A legacy system is typically a very old and large system which has been modified heavily since it started operation. Legacy systems are based on old technology with very little or no documentation. Dealing with a legacy system can be very hard.

Solutions for the problems mentioned above relating to a Legacy System:

- Explore possibility of subcontracting the maintenance
- Replace software with a package
- Re-implement from scratch
- Discard software and discontinue
- Freeze maintenance and phase in new system
- Reverse engineer the legacy system and develop a new software suite